Uma Revisão de Literatura sobre a Engenharia de Contexto e Orquestração de Agentes para Geração de Software

Luiz Vinícius Pereira de Oliveira Souza

[1]Departamento de Ciências Exatas e da Terra, Campus I
Universidade do Estado da Bahia (UNEB)
Salvador, Bahia, Brasil

luizvinicius128@gmail.com

Resumo. *A construção de Minimum Viable Products (MVPs) é uma estratégia fundamental para a validação rápida de requisitos e modelos de negócio nas fases iniciais do ciclo de desenvolvimento de software. Nesse cenário, a integração de Large Language Models (LLMs) desponta como uma força transformadora, acelerando drasticamente a prototipação e a geração automatizada desses artefatos. No entanto, a delegação dessa tarefa a abordagens lineares, baseadas exclusivamente em instruções em linguagem natural, cria um grave problema de controle e previsibilidade sobre o sistema construído. A ausência de restrições de engenharia rigorosas frequentemente resulta em "alucinações arquiteturais" e na geração de código não aderente aos padrões de produção, culminando em um rápido e severo acúmulo de dívida técnica. Para mitigar essas problemáticas, este artigo apresenta uma Revisão Sistemática da Literatura focada na interseção entre Engenharia de Contexto, Retrieval-Augmented Generation (RAG) e a orquestração de agentic workflows. Conclui-se que o uso do paradigma LLM-as-a-Judge como mecanismo de bloqueio e avaliação contínua, aliado a validações Human-in-the-Loop, é essencial para garantir a coesão estrutural de MVPs gerados por inteligência artificial. Dessa forma, esta revisão sintetiza como a literatura atual responde aos desafios de alinhamento e qualidade de código, mapeando as práticas de engenharia de contexto e orquestração necessárias para a geração de software guiada por restrições.*

Abstract. *Building Minimum Viable Products (MVPs) is a fundamental strategy for the rapid validation of requirements and business models in the early stages of the software development lifecycle. In this context, the integration of Large Language Models (LLMs) emerges as a transformative force, drastically accelerating prototyping and the automated generation of these artifacts. However, delegating this task to linear approaches based solely on natural language instructions creates a severe problem regarding the control and predictability of the built system. The lack of rigorous engineering constraints frequently results in "architectural hallucinations" and the generation of code that fails to adhere to production standards, culminating in a rapid and severe accumulation of technical debt. To mitigate these issues, this paper presents a Systematic Literature Review focusing on the intersection of Context Engineering, Retrieval-Augmented Generation (RAG), and the orchestration of agentic workflows. We conclude that employing the LLM-as-a-Judge paradigm as a continuous evaluation and gatekeeping mechanism, coupled with Human-in-the-Loop validations,*

is essential to ensure the structural cohesion of MVPs generated by artificial intelligence. Consequently, this review synthesizes how current literature addresses code alignment and quality challenges, mapping the context engineering and orchestration practices required for constraint-guided software generation.

1. Introdução

A construção de MVPs (*Minimum Viable Products*) consolidou-se como uma estratégia fundamental na engenharia de software para a validação ágil de requisitos e modelos de negócio nas fases iniciais do ciclo de desenvolvimento. Nesse cenário, a automação do processo de prototipação tem passado por uma transformação profunda com a integração de LLMs (*Large Language Models*) e agentes de Inteligência Artificial (IA). A crescente capacidade desses modelos de compreender instruções em linguagem natural e traduzi-las em código acelerou drasticamente a geração automatizada desses artefatos. Entre as abordagens emergentes, os *workflows* baseados em agentes autônomos e colaborativos ganham destaque por sua habilidade em não apenas sugerir trechos de código de forma isolada, mas por atuar ativamente na síntese de arquiteturas complexas [Calamo et al. 2026].

Apesar desses avanços, a análise de qualidade de código gerado por IA e a garantia de sua *production-readiness* (prontidão para produção) representam alguns dos maiores desafios da engenharia de software contemporânea. A geração de código em abordagens puramente lineares ou de *one-shot* (tentativa única) frequentemente resulta em "alucinações arquiteturais" e na propagação de erros cumulativos. Além disso, gera um acúmulo severo de dívidas técnicas específicas dessas tecnologias, fenômeno recentemente catalogado como *PromptDebt* [Aljohani and Do 2025]. A simples geração de uma sintaxe correta não garante que o software atenda a regras de negócio e restrições estruturais rigorosas, evidenciando a fragilidade de integrações de LLMs que carecem de especificações de interface claras [Shao et al. 2025].

Nesse cenário, torna-se imperativo o uso da Engenharia de Contexto — envolvendo estratégias como o RAG (*Retrieval-Augmented Generation*) [Curto 2025] e a injeção estruturada de restrições — aliada à orquestração de fluxos iterativos que utilizam o ser humano no ciclo de validação (*Human-in-the-Loop*) para alinhar a semântica do código aos requisitos reais [Zhang et al. 2025].

O objeto de estudo desta Revisão Sistemática da Literatura concentra-se na interseção entre a Engenharia de Contexto e a orquestração de *agentic workflows* (fluxos de trabalho agênticos) aplicados à inicialização de projetos de software. Esta pesquisa visa mapear o estado da arte das ferramentas e métodos utilizados para assegurar que os MVPs gerados por IA sejam estruturalmente coesos, aderentes a restrições de design e confiáveis para ambientes de produção. Para guiar a investigação, foram estabelecidas as seguintes Questões de Pesquisa (RQs):

- **RQ1:** Quais métodos e ferramentas de engenharia de contexto são atualmente utilizados para estabelecer restrições arquiteturais no desenvolvimento de software assistido por IA?
- **RQ2:** Como o uso de contextos estruturados impacta o alinhamento dos agentes com as restrições de produção e a redução de alucinações arquiteturais?

- **RQ3:** Quais métricas e métodos de avaliação são empregados na literatura para aferir a qualidade do código gerado por agentes baseados em LLMs?
- **RQ4:** Como os *frameworks* atuais orquestram *workflows* (ex: validação iterativa, *Human-in-the-Loop*) para garantir a confiabilidade do código gerado?

2. Relato da Revisão de Literatura

Nesta revisão sistemática, foi aplicado o filtro de disponibilidade do texto completo e um recorte temporal rigoroso (2023 a 2026) para focar no estado da arte e nos avanços recentes em tecnologias de inteligência artificial generativa. As bases de dados selecionadas para a busca foram ACM Digital Library, Springer Link, IEEE Digital Library e Scopus, devido à sua reconhecida abrangência e relevância nos campos de Engenharia de Software e Inteligência Artificial.

Para direcionar a pesquisa, os termos foram estruturados em quatro eixos principais: tecnologia base (LLMs e Agentes), intervenção (Engenharia de Contexto e RAG), domínio de aplicação (Desenvolvimento de Software e MVPs) e os resultados esperados (Qualidade de Código e Aderência Arquitetural). A composição da *string* de busca final, incluindo seus respectivos sinônimos, ficou definida da seguinte forma:

```
("LLM" OR "LLMs" OR "AI Agent" OR "AI Agents" OR
"Autonomous Agent" OR "Coding Agent" OR "Coding
assistant" OR "Large Language Model")
AND ("Context Engineering" OR "Architectural
constraints" OR "Context Injection" OR "Context
Management" OR "Design constraints" OR "Prompt
augmentation" OR "Repository context" OR "RAG" OR
"Retrieval-Augmented Generation" OR "Retrieval
Augmented Generation")
AND ("Software Development" OR "Software Engineering"
OR "MVP" OR "Minimum Viable Product" OR "Project
bootstrapping" OR "Project initialization" OR
"prototyping")
AND ("Code Quality" OR "Agent alignment" OR
"Architectural hallucination" OR "Constraint
adherence" OR "Production readiness" OR
"Production-ready")
```

A execução desta *string* nas bases de dados selecionadas resultou em um total inicial de 231 artigos. Após o processo de identificação e remoção de duplicatas, restaram 222 estudos únicos para a fase de triagem. A distribuição dos artigos retornados evidenciou uma forte concentração na ACM Digital Library (52,8%) e Springer Link (31,2%), seguidas pela IEEE Digital Library (12,1%) e Scopus (3,9%). Para assegurar o rigor metodológico e garantir que apenas os artigos alinhados aos objetivos da pesquisa fossem considerados para leitura completa, foram estabelecidos critérios claros de inclusão e exclusão, detalhados na Tabela 1.

Tabela 1. Critérios de inclusão e exclusão definidos para a triagem.

Identificador	Critério
CI1	Aborda estratégias de Engenharia de Contexto (ex: RAG, injeção de restrições, *System Prompts*).
CI2	Analisa orquestração de *Agentic Workflows* ou validação iterativa.
CI3	Avalia a aderência arquitetural, qualidade estrutural ou *production-readiness* de código gerado.
CI4	Foco central em Agentes de IA autônomos na geração de código-fonte, MVPs ou inicialização de projetos.
CE1	Desvio de domínio (Aplicação principal fora da Engenharia de Software).
CE2	Foco exclusivo no treinamento ou *Fine-Tuning* de modelos, sem abordar inferência ou orquestração.
CE3	Ferramentas restritas à assistência passiva ou *autocomplete* de linha única (sem fluxo agêntico).
CE4	Abordagens em plataformas *No-Code* ou *Low-Code* que não geram código-fonte bruto via LLMs.
CE5	Tipo de documento inadequado (estudos secundários, editoriais, *keynotes*, artigos curtos < 5 páginas).
CE6	Trabalhos incompletos (*Work in Progress*, resumos estendidos) sem avaliação empírica.
CE7	Idioma de publicação diferente do Inglês.
CE8	Texto completo (*full-text*) inacessível nas bases de dados.

Os quatro critérios de inclusão (CI1 a CI4) funcionaram como filtros positivos para selecionar artigos que propõem ou avaliam sistemas baseados em agentes e gerenciamento avançado de contexto para geração de código. Em contrapartida, os oito critérios de exclusão (CE1 a CE8) foram rigorosamente aplicados para eliminar estudos fora do domínio de software, focados apenas em autocompletar código de forma passiva, ou que não apresentassem maturidade acadêmica suficiente para fundamentar análises arquiteturais.

3. Resultados Parciais

Após a aplicação da *string* de busca e a filtragem inicial pela leitura de títulos e resumos sob a lente dos critérios de inclusão e exclusão, um conjunto restrito de estudos foi selecionado para leitura na íntegra. A Figura 1 apresenta a distribuição quantitativa dos artigos selecionados para a fase de leitura completa em contraste com os que foram efetivamente aceitos e compõem o portfólio final, agrupados por base de dados.

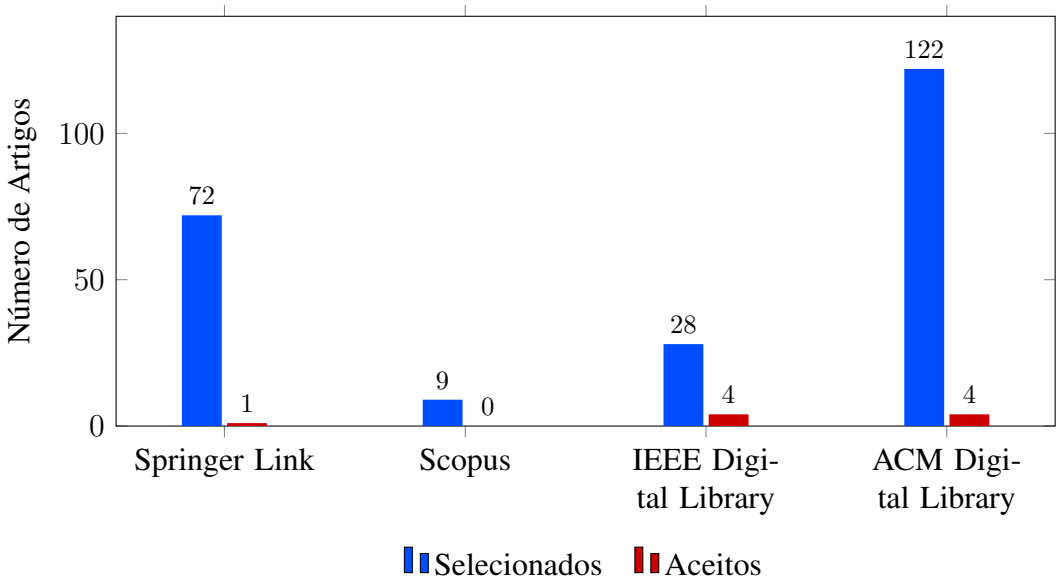


Figura 1. Distribuição de artigos selecionados versus aceitos por base de dados.

Pode-se observar que, embora a ACM Digital Library e a Springer Link tenham retornado o maior volume inicial de estudos selecionados, o rigor do escopo da pesquisa reduziu significativamente o número de aceites finais. Ao todo, 9 artigos avançaram para a etapa de Avaliação de Qualidade (QA).

A Avaliação de Qualidade foi conduzida com base em um questionário projetado para mensurar a profundidade metodológica dos artigos. Os critérios analisados incluíram: a presença de métricas claras para avaliação arquitetural, o detalhamento da Engenharia de Contexto, a orquestração de workflows, a robustez da validação empírica e a disponibilização de artefatos suficientes para replicação. A pontuação de corte (*cutoff score*) adotada foi permissiva, sendo estabelecida em 1,0/5,0. A Tabela 2 resume os resultados desta etapa. As linhas destacadas em verde representam os estudos que atingiram ou superaram a pontuação mínima, enquanto a linha em vermelho indica a exclusão do respectivo artigo.

Tabela 2. Resultados da Avaliação de Qualidade (QA) dos estudos lidos na íntegra.

Título do Estudo	Pontuação (QA)
PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects [Aljohani and Do 2025]	2.5
Empowering Agile-Based Generative Software Development through Human-AI Teamwork [Zhang et al. 2025]	4.0
Agents4PLC: Automating Closed-loop PLC Code Generation and Verification in Industrial Control Systems using LLM-based Agents [Liu et al. 2026]	4.5
Development of Agentic Workflows with LangGraph for Software Development Life Cycle Automation [Mandulapalli et al. 2026]	0.5
LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead [He et al. 2026]	1.5
Building Software Functional Requirements Lists Using RAG with Distinct LLMs in Multiple Interactions [Curto 2025]	2.5
From User Stories to Architectures: Using LLM-powered Agents to Design and Improve Microservice-based Software [Calamo et al. 2026]	4.5
Causality-Aided Evaluation and Explanation of Large Language Model-Based Code Generation [Ji et al. 2025]	3.0
Are LLMs Correctly Integrated into Software Systems? [Shao et al. 2025]	2.5

A análise revela que as soluções focadas em implementar uma orquestração cíclica real e restrições arquiteturais consolidadas, como evidenciado no *Agents4PLC* [Liu et al. 2026] e na arquitetura do *ArchiLLM* [Calamo et al. 2026], alcançaram as pontuações máximas (4.5). Em contrapartida, o artigo *"Development of Agentic Workflows with LangGraph for Software Development Life Cycle Automation"* [Mandulapalli et al. 2026] obteve a pontuação mínima (0.5), enquadrando-se como uma exclusão técnica por configurar-se unicamente como um estudo de visão/conceitual. Embora cite ferramentas como o LangGraph, a ausência de construção de um orquestrador real, falta de integração com RAG e inexistência de validação empírica demonstraram sua incompatibilidade com os parâmetros de qualidade estipulados para a pesquisa.

3.1. Análise Qualitativa dos Resultados

Após a leitura na íntegra e a extração sistemática de dados dos 8 artigos aceitos, foi possível identificar padrões, lacunas e soluções consolidadas na literatura recente. A análise qualitativa a seguir sintetiza esses achados, agrupando-os em eixos temáticos que respondem diretamente às Questões de Pesquisa estabelecidas no protocolo desta revisão.

3.1.1. O Custo das Abordagens Lineares e a Dívida Técnica (RQ1 e RQ2)

Um achado fundamental na literatura é a ineficácia e o risco associados a abordagens de geração de código baseadas em fluxos lineares (*one-shot pipelines*). Aljohani e Do [Aljohani and Do 2025], em uma análise de mais de 93 mil repositórios, identificaram que a integração direta e estática de LLMs, muitas vezes mediada de forma ingênua por frameworks como o LangChain, é a principal causa do fenômeno denominado *Prompt-Debt*. A ausência de restrições bem definidas e a dependência de *prompts* frágeis (como *Few-Shot* mal formatados) resultam em alta degradação estrutural do código.

Esse cenário é agravado quando a integração de técnicas de RAG é feita sem rigor arquitetural. Shao et al. [Shao et al. 2025] catalogaram 18 padrões de falha em sistemas que utilizam LLMs e RAG, demonstrando que o desalinhamento de contexto — onde o texto recuperado do banco vetorial é injetado sem formatação estrutural — causa quebras de funcionalidade e vulnerabilidades de segurança em 77% das aplicações analisadas. Tais evidências atestam que a criação manual ou não estruturada de regras de contexto é manifestamente insuficiente para garantir a prontidão para produção.

3.1.2. Estratégias de Engenharia de Contexto para MVPs (RQ1 e RQ2)

Para mitigar as alucinações e garantir o alinhamento com os requisitos do software, os estudos apontam para o uso de Engenharia de Contexto estruturada antes mesmo da fase de codificação. O framework *ArchiLLM* [Calamo et al. 2026] defende que os agentes devem gerar e validar uma arquitetura de alto nível (como listas de microsserviços) a partir de requisitos textuais mínimos antes de escrever o código-fonte. Contudo, a análise profunda desta solução revela que ela ainda opera de forma predominantemente sequencial, carecendo de um mecanismo ativo para reverter e corrigir o fluxo caso a arquitetura sofra alucinações no meio do processo.

A resposta da literatura para a eficácia do contexto reside em abordagens de RAG com estado iterativo (*Stateful RAG*). Curto [Curto 2025] comprova empiricamente que o uso de múltiplos ciclos de retroalimentação — em que a saída refinada de uma iteração serve como vetor de conhecimento enriquecido para a próxima — aumenta substancialmente a precisão semântica e bloqueia a progressão de alucinações cognitivas.

3.1.3. Orquestração Agêntica e Fluxos Iterativos (RQ4)

A adoção de *closed-loops* (ciclos fechados) de validação consolida-se como o padrão-ouro na orquestração agêntica. O Artigo *Agents4PLC* [Liu et al. 2026] representa o estado da arte nesta revisão ao orquestrar múltiplos agentes especialistas (planejador, programador, validador e depurador) que operam de forma cíclica. Em vez de aceitar o primeiro código gerado, o fluxo força um refinamento automático caso o código falhe na verificação. Essa interrupção e autocorreção ativa suprem exatamente a lacuna de linearidade observada no *ArchiLLM*.

Além da validação autônoma, a colaboração com o usuário demonstra ser um fator decisivo para a confiabilidade. O modelo *AgileGen* [Zhang et al. 2025] implementa o

paradigma *Human-in-the-Loop* ao exigir a aprovação do usuário sobre os critérios de aceitação gerados pela IA antes de disparar a geração de código efetiva. Essa pausa estratégica atua como um mecanismo de controle de qualidade humano, evitando a acumulação de erros (*cumulative errors*) típica do desenvolvimento em cascata guiado por IA.

3.1.4. Métricas de Qualidade e o Paradigma LLM-as-a-Judge (RQ3)

No que tange aos métodos de validação para a qualidade de código, a literatura converge na transição das métricas tradicionais baseadas em léxico (como *Pass@k* e *BLEU*) para o paradigma *LLM-as-a-Judge*. O estudo de visão de He et al. [He et al. 2026] argumenta que métricas estáticas falham severamente ao avaliar atributos arquiteturais não funcionais, como legibilidade, coesão estrutural e adesão a princípios de arquitetura limpa. O estudo defende a avaliação guiada por referências (*Reference-guided evaluation*) através de *System Prompts* restritivos, injetando rubricas claras de avaliação para impedir que a IA faça julgamentos subjetivos ou não fundamentados.

Para elevar a precisão matemática e lógica desse juiz autônomo, Ji et al. [Ji et al. 2025] introduzem a inferência causal na avaliação de código (*Causal-Eval*). O *insight* fundamental deste estudo para a engenharia de *workflows* é que o nó de julgamento não deve apenas reprovar uma arquitetura devolvendo um falso/verdadeiro. Ele deve mapear causalmente o erro: identificar qual restrição exata injetada no contexto foi desrespeitada e devolver essa relação de causa e efeito como *feedback* restritivo e acionável para o agente gerador na iteração seguinte, fechando, assim, o ciclo de validação de forma determinística.

4. Conclusões

A condução desta Revisão Sistemática da Literatura evidenciou uma transição paradigmática no desenvolvimento de software assistido por IA: a evolução da simples geração probabilística de trechos de código para a síntese arquitetural rigorosa guiada por contexto. Conclui-se que a autonomia eficaz das LLMs no desenvolvimento de software é, paradoxalmente, dependente da imposição de restrições rígidas. A confiabilidade e a *production-readiness* de MVP gerado por IA não derivam apenas da capacidade bruta do modelo fundacional escolhido, mas fundamentalmente do rigor da Engenharia de Contexto aplicada e do *design* iterativo do fluxo de orquestração.

Apesar da consolidação teórica de conceitos como RAG de estado iterativo e do paradigma de avaliação *LLM-as-a-Judge*, a literatura analisada ainda apresenta lacunas tecnológicas e metodológicas expressivas. Cabe ressaltar que os resultados aqui obtidos estão circunscritos às quatro bases de dados selecionadas e ao recorte temporal recente, o que pode representar uma limitação em relação à abrangência total do campo. Contudo, uma limitação crítica e latente identificada na área é a carência de infraestruturas *open-source* que implementem fluxos com *closed-loop* focados estritamente na validação técnica de restrições de engenharia. Constatou-se que muitas ferramentas que se intitulam "agênticas" operam, sob o capô, como *pipelines* lineares vulneráveis, que acumulam *PromptDebt* e propagam alucinações estruturais irremediáveis na base de código. Além disso, o uso atual de *LLM-as-a-Judge* frequentemente se restringe a avaliações *post-hoc* para fins acadêmicos, falhando em atuar ativamente no tempo de execução com retornos

causais que corrijam o agente codificador antes da compilação final.

Com base nos desafios evidenciados, identifica-se uma clara oportunidade de pesquisa para o aprimoramento das abordagens de orquestração atuais. Diante das vulnerabilidades dos modelos lineares, o anteprojeto atrelado a este trabalho tem como objetivo explorar a viabilidade prática de um orquestrador fundamentado em máquinas de estado dinâmicas (através de *frameworks* como LangGraph), visando uma geração de software mais resiliente e controlada.

Referências

Aljohani, A. and Do, H. (2025). Promptdebt: A comprehensive study of technical debt across llm projects. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, EASE '25, page 371–382, New York, NY, USA. Association for Computing Machinery.

Calamo, M., Monti, F., Spaziani, F., Leotta, F., and Mecella, M. (2026). From user stories to architectures: Using llm-powered agents to design and improve microservice-based software. *IEEE Software*, pages 1–8.

Curto, H. N. (2025). Building software functional requirements lists using rag with distinct llms in multiple interactions. In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, pages 612–616.

He, J., Shi, J., Zhuo, T. Y., Treude, C., Sun, J., Xing, Z., Du, X., and Lo, D. (2026). Llm-as-a-judge for software engineering: Literature review, vision, and the road ahead. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.

Ji, Z., Ma, P., Li, Z., Wang, Z., and Wang, S. (2025). Causality-aided evaluation and explanation of large language model-based code generation. *Proc. ACM Softw. Eng.*, 2(ISSTA).

Liu, Z., Zeng, R., Wang, D., Peng, G., Liu, X., Liu, Q., Liu, P., Wang, W., and Wang, J. (2026). Agents4plc: Automating closed-loop plc code generation and verification in industrial control systems using llm-based agents. *IEEE Transactions on Software Engineering*, pages 1–16.

Mandulapalli, S., Hernandez, E., Hall, W. J., Chakeri, A., and Jaimes, L. (2026). Development of agentic workflows with langgraph for software development life cycle automation. In Florez, H., Rabelo, L., and Diaz, C., editors, *Industrial Engineering and Operations Management*, pages 45–54, Cham. Springer Nature Switzerland.

Shao, Y., Huang, Y., Shen, J., Ma, L., Su, T., and Wan, C. (2025). Are llms correctly integrated into software systems? In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ICSE '25, page 1178–1190. IEEE Press.

Zhang, S., Xing, Z., Guo, R., Xu, F., Chen, L., Zhang, Z., Zhang, X., Feng, Z., and Zhuang, Z. (2025). Empowering agile-based generative software development through human-ai teamwork. *ACM Trans. Softw. Eng. Methodol.*, 34(6).

5. Planilha-resumo de Resultados

Tabela 3. Planilha-resumo dos trabalhos selecionados e aceitos.

Referência	CI1	CI2	CI3	CI4	Descrição da Extração	Avaliação
Aljohani and Do (2025)	X			X	Estudo empírico/mineração de dados que analisa 93.142 repositórios. Identifica que integrações frágeis de frameworks como LangChain e *prompts* de instrução/*Few-Shot* mal formatados geram o fenômeno catalogado como *PromptDebt*.	Fornece a base empírica de que integrações lineares causam degradação estrutural, justificando a necessidade de orquestração iterativa e injeção rigorosa de restrições arquiteturais.
Shao et al. (2025)	X			X	Avalia a integração de RAG em 100 sistemas *open-source*. Mapeia 18 padrões de falha (*Integration Defects*), mostrando que 77% das aplicações falham por desalinhamento de contexto na injeção do prompt.	Demonstra que o RAG sem filtragem estrita ou validação estrutural injeta "ruído", comprovando a fragilidade das buscas vetoriais cegas em sistemas lineares.
Calamo et al. (2026)	X			X	Apresenta o framework *ArchiLLM*, que atua na arquitetura de sistema (*System Design*) injetando restrições visando a prontidão para produção (*Production-readiness*).	Mostra a importância de desenhar a arquitetura antes da codificação, mas evidencia a limitação de abordagens predominantemente lineares que carecem de *feedback* causal.
Curto (2025)	X	X			Estudo empírico focando na elicitação de requisitos. Utiliza um *Stateful RAG*, onde os requisitos gerados são retroalimentados na base vetorial iterativamente, com um *Human-in-the-Loop* definindo o fim do ciclo.	Comprova que ciclos iterativos de validação, retroalimentando o estado (como no LangGraph), aumentam a precisão semântica e reduzem a alucinação das respostas da IA.

Continua na próxima página

Tabela 3 – continuação da página anterior

Referência	CI1	CI2	CI3	CI4	Descrição da Extração	Avaliação
Liu et al. (2026)	X	X	X	X	Apresenta o *Agents4PLC*, um sistema multiagente focado em controle industrial. Implementa uma arquitetura de malha fechada (*closed-loop*), forçando a interrupção e a regeneração de código se a validação falhar.	Representa o estado da arte na orquestração ao aplicar um ciclo contínuo de autodepuração, preenchendo exatamente a lacuna de linearidade observada em frameworks mais simples.
Zhang et al. (2025)		X		X	O framework *AgileGen* aplica metodologias ágeis e *Human-in-the-Loop*, exigindo aprovação manual dos critérios de aceite antes de prosseguir com a geração do código.	Evidencia que paradas estratégicas no *pipeline* (*gatekeepers*) bloqueiam o avanço de alucinações cognitivas e impedem o acúmulo de erros no ciclo de vida.
He et al. (2026)	X		X		Revisão e documento de visão estabelecendo formalmente o paradigma *LLM-as-a-Judge*. Alerta que a métrica estática é insuficiente e prescreve o uso de *System Prompts* restritivos para a avaliação.	Justifica teoricamente o uso de LLMs para avaliar critérios não-funcionais (como arquitetura limpa), fundamentando o nó de julgamento da arquitetura do projeto.
Ji et al. (2025)	X	X	X		Propõe o framework *Causal-Eval*. Instrui o modelo a não apenas julgar o código, mas mapear a causalidade do erro, identificando exatamente qual parte do *prompt* inicial falhou.	Fornece o modelo matemático para programar o juiz autônomo, capacitando-o a retornar *feedbacks* restritivos e acionáveis para o agente gerador nas iterações de malha fechada.